Week 48

v1.0.0

Table of contents

1 Enzyme inhibitors (I) 2
2 Enzyme inhibitors (II) 7
3 Enzymatic behaviour of the enzyme ATCase 13

```
try:
    import fysisk_biokemi
    print("Already installed")
except ImportError:
    %pip install -q "fysisk_biokemi[pyodide] @ https://au-mbg.github.io/fysisk-biokemi/
wheels/fysisk_biokemi-0.2.2-py3-none-any.whl"
```

---

1 Enzyme inhibitors (I)

```
import matplotlib.pyplot as plt
import pandas as pd
from scipy.optimize import curve_fit
import numpy as np
from IPython.display import display
pd.set_option('display.max_rows', 6)
```

An enzyme obeying the Michaelis-Menten kinetics model was tested for substrate conversion in the absence and presence of an inhibitor, called inhibitor1 at a concentration of $[\mathrm{I}] = 2.5 \cdot 10^{-3}$ M. The data set is contained in the file `enzyme-inhib-i.xlsx`. Using this data a researcher wanted to determine the type of inhibition.

Start by loading the dataset

```
from IPython.display import display
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('week49_6') # Load from package for the solution so it doesn't require to
interact.
display(df)
```

	[S]_(mM)	V0_no_inhib_(uM/s)	V0_inhib_(uM/s)
0	1	15.2	5.2
1	1	15.1	5.2
2	1	14.5	5.0
…	…	…	…
36	50	60.7	50.5
37	50	61.4	50.5
38	50	60.8	52.0

(a) Convert units

Convert the concentrations of substrate and the initial velocities to units given in M and $\mathrm{M} \cdot \mathrm{s}^{-1}$, respectively.

```
## Your task: Make new columns with the properties in SI units.
df['[S]_(M)'] = df['[S]_(mM)'] * 10**(-3)
df['V0_no_inhib_(M/s)'] = df['V0_no_inhib_(uM/s)'] * 10**(-6)
df['V0_inhib_(M/s)'] = df['V0_inhib_(uM/s)'] * 10**(-6)
display(df)
```

	[S]_(mM)	V0_no_inhib_(uM/s)	V0_inhib_(uM/s)	[S]_(M)	V0_no_inhib_(M/s)	V0_inhib_(M/s)
0	1	15.2	5.2	0.001	0.000015	0.000005
1	1	15.1	5.2	0.001	0.000015	0.000005
2	1	14.5	5.0	0.001	0.000015	0.000005

	[S]_(mM)	V0_no_inhib_(uM/s)	V0_inhib_(uM/s)	[S]_(M)	V0_no_inhib_(M/s)	V0_inhib_(M/s)
...	...	...	...	...	...	...
36	50	60.7	50.5	0.050	0.000061	0.000051
37	50	61.4	50.5	0.050	0.000061	0.000051
38	50	60.8	52.0	0.050	0.000061	0.000052

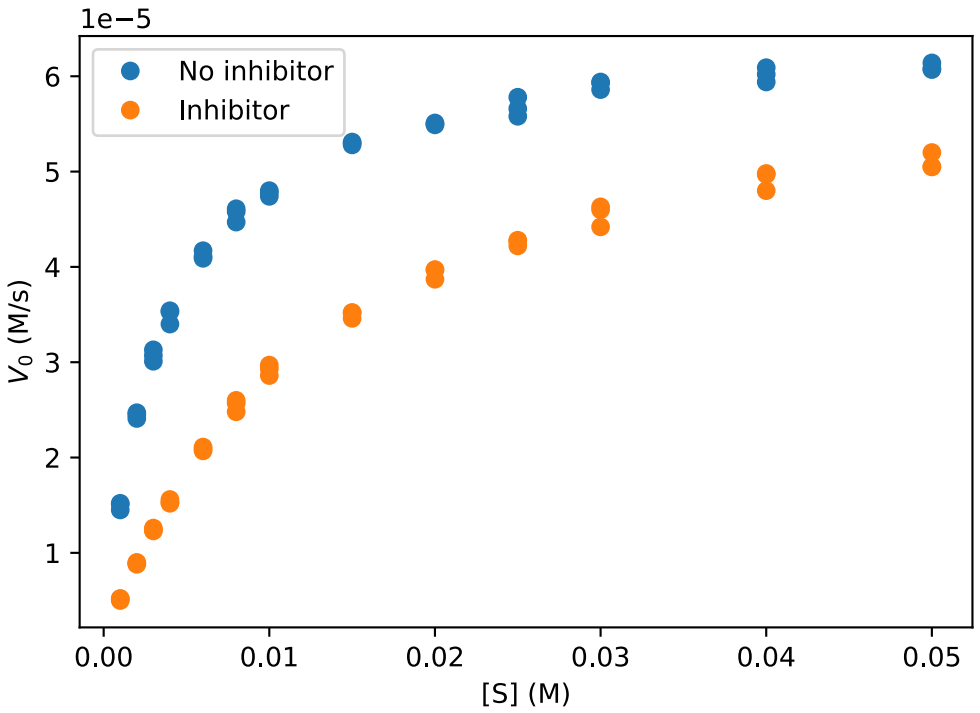
(b) Plot

Plot the initial velocities of both experiments as a function of substrate concentration in one plot. Estimate K_M and $V_{\max}$ in the presence and absence of inhibitor from the plot.

```
fig, ax = plt.subplots(figsize=(6, 4))

## Your task: Plot the initial velocities for both experiments.
ax.plot(df['[S]_(M)'], df['V0_no_inhib_(M/s)'], 'o', label='No inhibitor')
ax.plot(df['[S]_(M)'], df['V0_inhib_(M/s)'], 'o', label='Inhibitor')

## Sets the x and y-axis labels and shows the legend.
ax.set_xlabel('[S] (M)')
ax.set_ylabel('$V_0$ (M/s)')
ax.legend()
```



Use the plots to estimate K_M and $V_{\max}$ and assign in the cell below

```
## Your task: Estimate parameters without inhibitor.
K_M_no_inhib_estimate = 0.0025 # Unit: M
Vmax_no_inhib_estimate = 6 * 10**(-5) # Unit: M/s

## Your task: Estimate parameters with inhibitor.
K_M_inhib_estimate = 0.01 # Unit: M
Vmax_inhib_estimate = 6 * 10**(-5) # Unit: M/s
```

(c) Fit

The researcher wanted to determine $V_{\max}$ and K_M values for both experiments in order to correctly conclude on the type of inhibitor.

Determine $V_{\max}$ and K_M by fitting.

Start writing the function to fit with

```
## Your task: Implement the Michealis-Menten equation as a function
def michaelis_menten(S, Vmax, Km):
    return (Vmax * S) / (S + Km)
```

And use the `curve_fit`-function to find the parameters

```
## Your task: Make a fit to the Michealies-Menten equation for the data without the
inhibitor.
initial_guess = [Vmax_no_inhib_estimate, K_M_no_inhib_estimate]
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['V0_no_inhib_(M/
s)'], initial_guess)
Vmax_fit_no_inhib, K_M_fit_no_inhib = fitted_parameters

## Your task: Make a fit to the Michealies-Menten equation for the data the inhibitor.
initial_guess = [Vmax_inhib_estimate, K_M_inhib_estimate]
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['V0_inhib_(M/s)'],
initial_guess)
Vmax_fit_inhib, K_M_fit_inhib = fitted_parameters

## Printing
print('Without inhibitor:')
print('    V_max:', Vmax_fit_no_inhib)
print('    K_M:', K_M_fit_no_inhib)
print('With inhibitor:')
print('    V_max:', Vmax_fit_inhib)
print('    K_M:', K_M_fit_inhib)
```

```
Without inhibitor:
    V_max: 6.495499633641784e-05
    K_M: 0.003410851437470283
With inhibitor:
    V_max: 6.3730854803554e-05
    K_M: 0.01220088316450348
```

Which of the kinetic parameters changes substantively in the presence of the inhibitor

! Answer

K_M

(d) Analyze the fit

As per usual, we need to plot the fit to confirm that that it has been done successfully.

We start by evaluating the fit for each set of parameters - those with and those without the inhibitor.

```
## Your task: Evaluate the fits for both cases
S_smooth = np.linspace(0.0, 0.055)
V0_no_inhib_model = michaelis_menten(S_smooth, Vmax_fit_no_inhib, K_M_fit_no_inhib)
V0_inhib_model = michaelis_menten(S_smooth, Vmax_fit_inhib, K_M_fit_inhib)
```

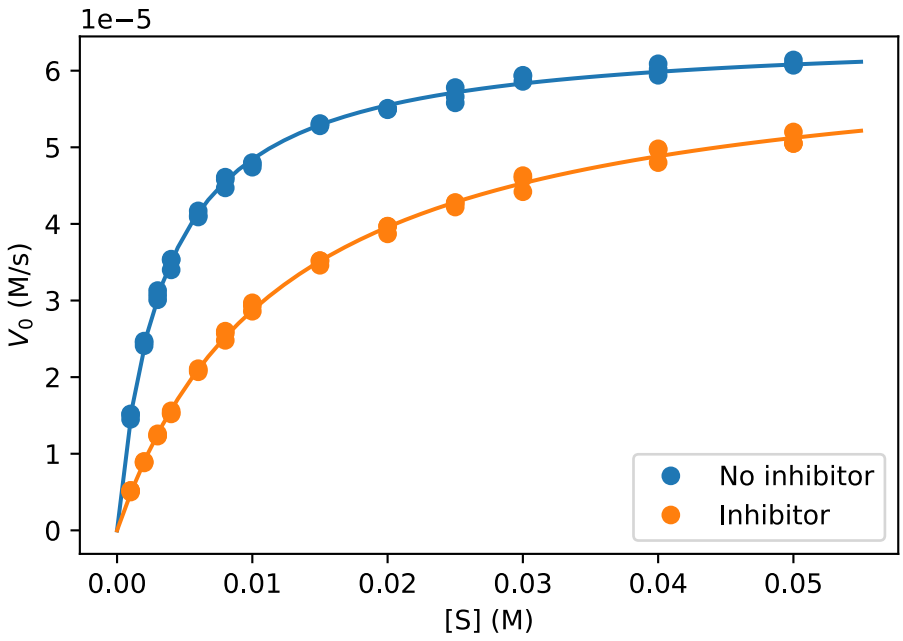
Copy your code from above and then we plot the fit along with the data,

```
fig, ax = plt.subplots()

## Your task: Plot the fits:
ax.plot(S_smooth, V0_no_inhib_model)
ax.plot(S_smooth, V0_inhib_model)

## Plots the data:
ax.plot(df['[S]_(M)'], df['V0_no_inhib_(M/s)'], 'o', label='No inhibitor', color='C0')
ax.plot(df['[S]_(M)'], df['V0_inhib_(M/s)'], 'o', label='Inhibitor', color='C1')

## Sets the x and y-axis labels and shows the legend.
ax.set_xlabel('[S] (M)')
ax.set_ylabel('$V_0$ (M/s)')
ax.legend()
```



(e) Inhibitor type

What type of inhibitor is inhibitor1?

! Answer

As $V_{\max}$ is approximately the same but K_M is significantly increased with the inhibitor that suggests a **competitive** inhibitor.

(f) K_i

The experiment was done in the presence of 2.5 mM of the inhibitor. What is the K_i for this inhibitor?

```
## Your task: Set the inhibitor concentration in M.
I_concentration = 2.5e-3  # M

## Your task: Calculate K_i from the fitted K_M values
K_i = I_concentration / ((K_M_fit_inhib / K_M_fit_no_inhib) - 1)

print(K_i)
```

```
0.000970090764001574
```

---

2 Enzyme inhibitors (II)

```
import matplotlib.pyplot as plt
import pandas as pd
from scipy.optimize import curve_fit
import numpy as np
from IPython.display import display
pd.set_option('display.max_rows', 6)
```

Two enzyme inhibitors were identified as part of a drug discovery program. To characterize the mechanism of action, the reaction kinetics were analysed for the enzyme alone and in the presence of 5 μM of each of the inhibitors as a function of substrate concentration resulting in the data in the file `enzyme-inhib-ii.xlsx`

Load the dataset

```
from IPython.display import display
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('week49_7') # Load from package for the solution so it doesn't require to
interact.
display(df)
```

	[S]_(uM)	enz_(nM/s)	inhibitor2_(nM/s)	inhibitor3_(nM/s)
0	0.2	0.070	0.031	0.021
1	0.2	0.067	0.029	0.022
2	0.2	0.071	0.029	0.025
...	...	...	...	...
30	20.0	0.312	0.040	0.119
31	20.0	0.309	0.036	0.113
32	20.0	0.314	0.045	0.120

(a) Convert & plot

Convert the measurements to SI units and plot the initial reaction velocity versus substrate concentration for all three reactions.

Start by converting the units

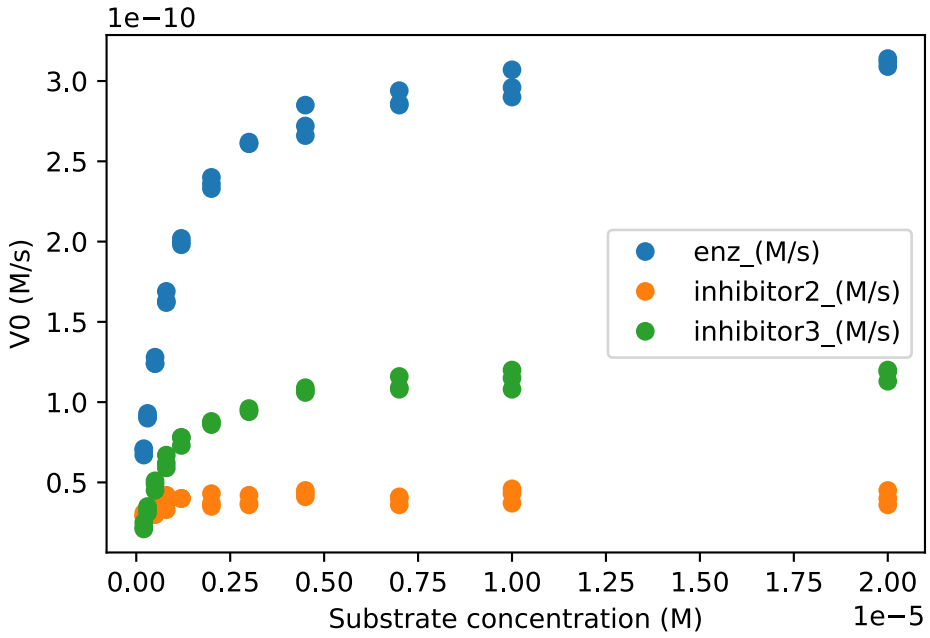
```
## Your task: Add new columns with the properties in the correct SI units.
## There are 4 properties, so you should add 4 columns.
df['[S]_(M)'] = df['[S]_(uM)'] * 10**(-6)
df['enz_(M/s)'] = df['enz_(nM/s)'] * 10**(-9)
df['inhibitor2_(M/s)'] = df['inhibitor2_(nM/s)'] * 10**(-9)
df['inhibitor3_(M/s)'] = df['inhibitor3_(nM/s)'] * 10**(-9)
```

And then plot to visualize the data

```
fig, ax = plt.subplots()

## Your task plot each data vs. [S]_(M).
ax.plot(df['[S]_(M)'], df['enz_(M/s)'], 'o', label='enz_(M/s)')
ax.plot(df['[S]_(M)'], df['inhibitor2_(M/s)'], 'o', label='inhibitor2_(M/s)')
ax.plot(df['[S]_(M)'], df['inhibitor3_(M/s)'], 'o', label='inhibitor3_(M/s)')

## Adds legend and axis labels.
ax.legend()
ax.set_xlabel('Substrate concentration (M)')
ax.set_ylabel('V0 (M/s)')
plt.show()
```



(b) Estimate parameters

Based on the appearance of these plots:

- Which dataset would have the highest $V_{\max}$, which the lowest?
- What about K_M?

Based on your answers,

- Can you determine the type of inhibition?
- Alternatively, can you exclude some mechanisms?

! Answer

$V_{\max}$ changes by a lot, with little or no change in K_M. This rules out competetive inhibition.

(c) Fit

As always, we need the function we're fitting with

```
def michaelis_menten(S, Vmax, Km):
    return Vmax * S / (S + Km)
```

And then we can make the fit

```
## Your task fit dataset 1: 'enz_(M/s)'
initial_guess = [3*10**(-9), 0.25*10**(-5)]
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['enz_(M/s)'],
initial_guess)
Vmax_enz_fit, K_M_enz_fit = fitted_parameters

## Your task fit dataset 2: 'inhibitor2_(M/s)'
initial_guess = [3*10**(-9), 0.25*10**(-5)]
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['inhibitor2_(M/
s)'], initial_guess)
Vmax_inhib2_fit, K_M_inhib2_fit = fitted_parameters

## Your task fit dataset 3: 'inhibitor2_(M/s)'
initial_guess = [3*10**(-9), 0.25*10**(-5)]
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['inhibitor3_(M/
s)'], initial_guess)
Vmax_inhib3_fit, K_M_inhib3_fit = fitted_parameters

print('Dataset 1: enz_(M/s)')
print('    V_max', Vmax_enz_fit)
print('    K_M', K_M_enz_fit)
print('Dataset 2: inhibitor2_(M/s)')
print('    V_max', Vmax_inhib2_fit)
print('    K_M', K_M_inhib2_fit)
print('Dataset 3: inhibitor3_(M/s)')
print('    V_max', Vmax_inhib3_fit)
print('    K_M', K_M_inhib3_fit)
```

```
Dataset 1: enz_(M/s)
    V_max 3.2194460122382126e-10
    K_M 7.505110277804259e-07
Dataset 2: inhibitor2_(M/s)
    V_max 4.0952644440159935e-11
    K_M 8.442553721173573e-08
Dataset 3: inhibitor3_(M/s)
    V_max 1.2316051281808638e-10
    K_M 7.913115764963332e-07
```

```
fig, ax = plt.subplots()

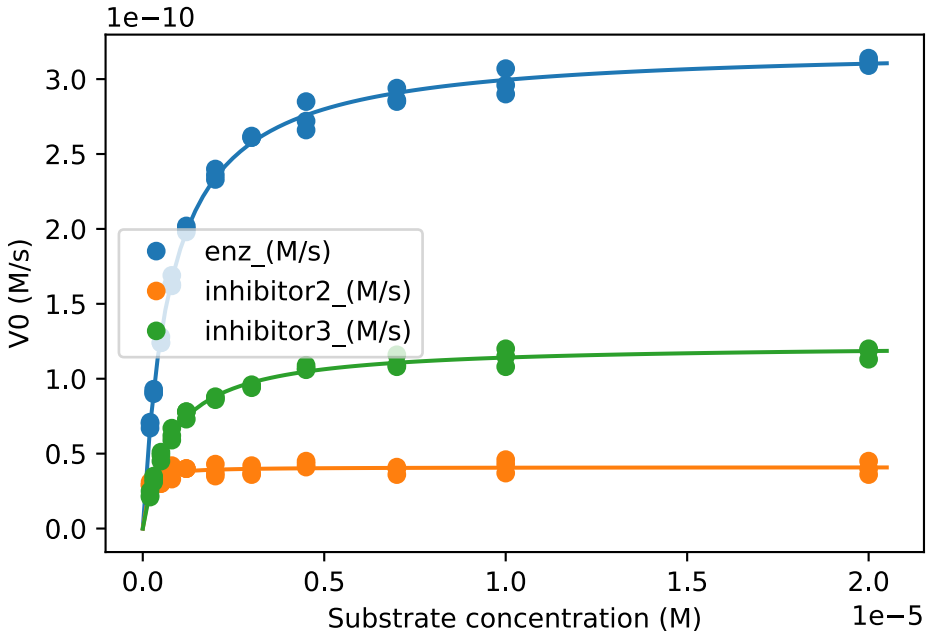
## Your task: Evaluate the fits
S_smooth = np.linspace(0, 2.05*10**(-5), 100)
```

```
V0_enz = michaelis_menten(S_smooth, Vmax_enz_fit, K_M_enz_fit)
V0_inhib2 = michaelis_menten(S_smooth, Vmax_inhib2_fit, K_M_inhib2_fit)
V0_inhib3 = michaelis_menten(S_smooth, Vmax_inhib3_fit, K_M_inhib3_fit)

## Plots the fits
ax.plot(S_smooth, V0_enz)
ax.plot(S_smooth, V0_inhib2)
ax.plot(S_smooth, V0_inhib3)

## Plots the data
ax.plot(df['[S]_(M)'], df['enz_(M/s)'], 'o', label='enz_(M/s)', color='C0')
ax.plot(df['[S]_(M)'], df['inhibitor2_(M/s)'], 'o', label='inhibitor2_(M/s)', color='C1')
ax.plot(df['[S]_(M)'], df['inhibitor3_(M/s)'], 'o', label='inhibitor3_(M/s)', color='C2')

## Adds legend and axis labels.
ax.legend()
ax.set_xlabel('Substrate concentration (M)')
ax.set_ylabel('V0 (M/s)')
plt.show()
```



If your fits don't closely match the data, go back and change your initial guesses.

(d) Type of inhibitor

We call $V_{\max}$ and K_{M} recorded in the presence of the inhibitor for $V_{\max}^{app}$ and K_{M}^{app} (app = apparent).

Calcuate how much each value changes in the presence of the inhibitor by dividing them by the parameters determined in the absence of an inhibitor.

```
## Your task: Calculate the ratio nof K_M^app / K_M
K_M_ratio2 = K_M_inhib2_fit / K_M_enz_fit
K_M_ratio3 = K_M_inhib3_fit / K_M_enz_fit

## Your task: Calculate the ratio nof Vmax^app / Vmax
Vmax_ratio2 = Vmax_inhib2_fit / Vmax_enz_fit
Vmax_ratio3 = Vmax_inhib3_fit / Vmax_enz_fit

## Prints the ratios
print('Inhibitor 2')
print('K_M', K_M_ratio2)
print('Vmax', Vmax_ratio2)

print('Inhibitor 3')
print('K_M', K_M_ratio3)
print('Vmax', Vmax_ratio3)
```

```
Inhibitor 2
K_M 0.11249073509474904
Vmax 0.12720401051760136
Inhibitor 3
K_M 1.0543636898135547
Vmax 0.3825518811308258
```

Based on the above, what is the likely inhibition type of inhibitor2 and inhibitor3?

! Answer

For inhibitor2 both K_M and $V_{\max}$ change significantly, making the likely type of inhibition **uncompetitive**.

For inhibitor3 only $V_{\max}$ changes significantly, making the likely type of inhibition **non-competiive**.

(e) K_i

Determine the K_i for each of the two inhibitors. The inhibitor concentration is 5 μM.

```
## Your task: Assign known value
I = 5 * 10**(-6)

## Your task: Calculate K_I for both cases.
## You should derive the correct expression before doing any coding!
K_I_inhib2 = I / (Vmax_enz_fit / Vmax_inhib2_fit - 1)
K_I_inhib3 = I / (Vmax_enz_fit / Vmax_inhib3_fit - 1)

## Prints the results
print('Inhibitor 2: ', K_I_inhib2)
print('Inhibitor 3: ', K_I_inhib3)
```

```
Inhibitor 2:  7.287155993523652e-07
Inhibitor 3:  3.0978463569655915e-06
```

! Answer

For both (pure) non-competitive and uncompetitive inhibition we have

$$V_{\max}^{app} = \frac{V_{\max}}{1 + \frac{[I]}{K_I}}.$$

From which we can isolate K_I, doing a bit of algebra we get

$$K_I = \frac{[I]}{\frac{V_{\max}}{V_{\max}^{app}} - 1}.$$

Also good practice to think about units, note that $V_{\max}$ and $V_{\max}^{app}$ have the same units ($\mathrm{M\,s^{-1}}$), so their ratio is dimensionless:

$$\frac{V_{\max}}{V_{\max}^{app}} \quad \text{units:} \quad \frac{\mathrm{M\,s^{-1}}}{\mathrm{M\,s^{-1}}} = 1.$$

Therefore the units of K_I are

$$K_I \quad \text{units:} \quad \frac{\mathrm{M}}{(1) - 1} \quad \Rightarrow \quad \mathrm{M}.$$

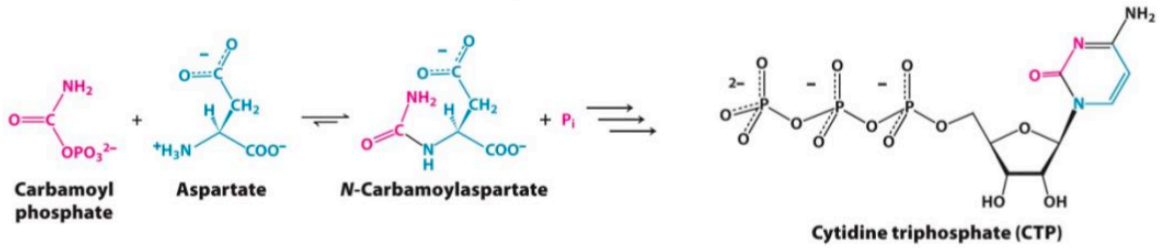
Which is the expected unit.

---

3 Enzymatic behaviour of the enzyme ATCase

```
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
import numpy as np
import pandas as pd
from IPython.display import display
```

The enzyme aspartate transcarbamoylase (ATCase) catalyzes the first reaction in the biosynthesis of pyrimidines such as CTP as shown in the reaction below:



ATCase does not obey the Michaelis-Menten kinetics model but instead shows the behaviour recorded in the `enzyme-behav-atcase.csv` dataset.

The dataset consists of three columns; the aspartate concentration in mM and the rate of formation of N-carbamyolaspartate with and without the presence of CTP.

Load the dataset with pandas. Set `data_file_path` to the path of the dataset file named above.

```
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('atcase') # Load from package for the solution so it doesn't require to
interact.
display(df)
```

	[aspartate]_(mM)	rate_(uM/s)	rate_ctp_(uM/s)
0	0.000000	−0.006903	0.005021
1	1.724138	0.330083	0.055910
2	3.448276	1.582132	0.404360
…	…	…	…
27	46.551724	17.641138	17.347028
28	48.275862	17.669610	17.393272
29	50.000000	17.679950	17.446557

(a) Units & Plot

```
## Your task: Add columns with the properties in SI units.
df['[aspartate]_(M)'] = df['[aspartate]_(mM)'] * 10**(-3)
```

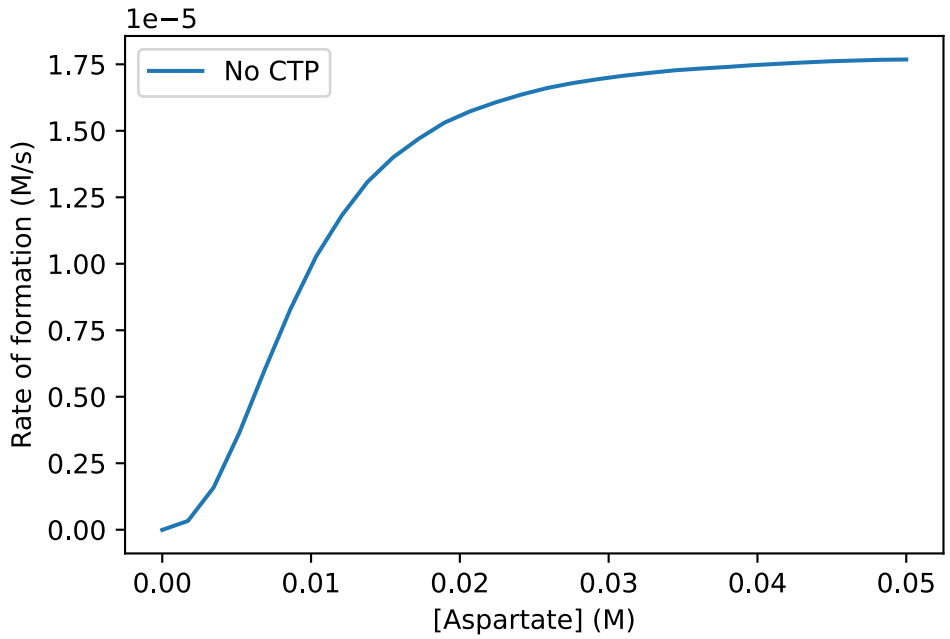
```
df['rate_(M/s)'] = df['rate_(uM/s)'] * 10**(-6)
df['rate_ctp_(M/s)'] = df['rate_ctp_(uM/s)'] * 10**(-6)
```

Make a plot of the `rate_(M/s)` dataset versus the aspartate concentration.

```
fig, ax = plt.subplots()

## Your task: Plot the datasets
ax.plot(df['[aspartate]_(M)'], df['rate_(M/s)'], label='No CTP')

ax.set_xlabel('[Aspartate] (M)')
ax.set_ylabel('Rate of formation (M/s)')
ax.legend()
```



(b) Kinetic profile

Looking at the curve without CTP, describe the kinetic profile of ATCase and explain what it tells us about the way ATCase works. (You may find inspiration in the material previously covered on protein-ligand interactions)

! Answer

The midpoint has a steep increase (large slope), whereas the starting and end points has smaller slopes. In other words, the reaction velocity V_0 shows sigmoidity as a function of substrate concentration, [aspartate].

This tells us that ATCase works by cooperativity in substrate binding (i.e. does not obey Michaelis-Menten kinetics model), i.e. binding of one susbtrate increases the likelihood that the second substrate will bind (you have already encountered this in the topic of protein-ligand interactions).

(c) Quarternary structure

What does the figure tell us about the quaternary structure of ATCase?

! Answer

Allosteric enzymes consist of two or more subunits, each able to bind and process a substrate molecule. Such enzymes can take at least two different conformational states, which are in equilibrium.

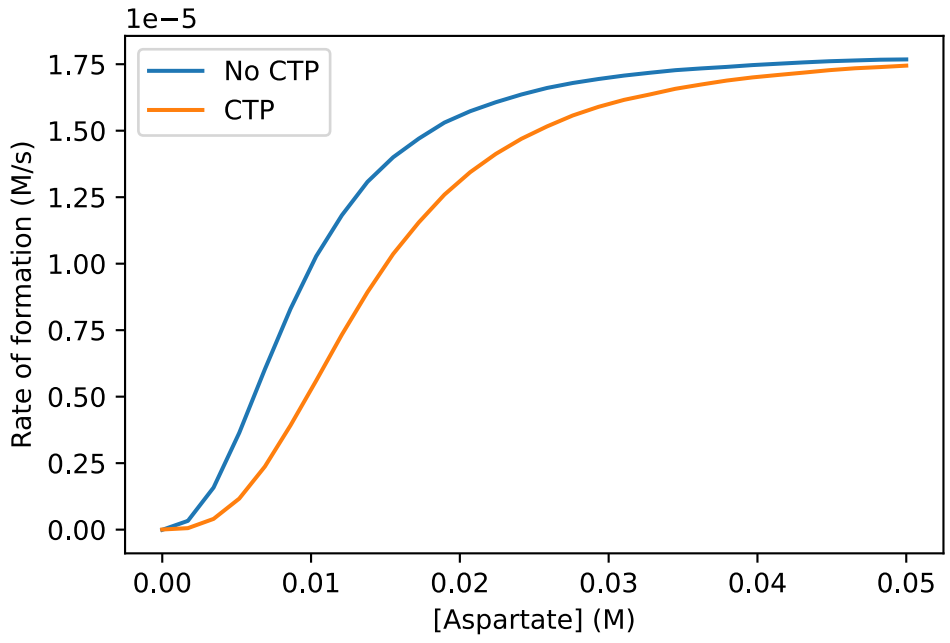
(d) Effect of CTP

Copy your plotting code from above and add a line to also plot the `rate_ctp_(M/s)` dataset.

```
fig, ax = plt.subplots()

## Your task: Plot the datasets
ax.plot(df['[aspartate]_(M)'], df['rate_(M/s)'], label='No CTP')
ax.plot(df['[aspartate]_(M)'], df['rate_ctp_(M/s)'], label='CTP')

ax.set_xlabel('[Aspartate] (M)')
ax.set_ylabel('Rate of formation (M/s)')
ax.legend()
```



Qualitatively describe the effect of CTP on the rate of N-carbamyolaspartate formation

! Answer

CTP lowers the catalytic rate of ATCase, by binding to it and stabilising ATCase in a low substrate-affinity state.

(e) Physiological advantage

In fact many enzymes are regulated by certain end products in a fashion similar to the CTP effect on ATCase. Can you explain why this might be a physiological advantage?

! Answer

As seen from the reaction scheme, the reaction catalyzed by ATCase ultimately leads to the formation of CTP. When [CTP] increases the level of CTP formation therefore drops.

(f) Regression with Hill equation.

There is also a formulation of the Hill equation for the enzyme kinetics, which has the following form:

$$v = \frac{V_{\max} \cdot S^h}{K_{\frac{1}{2}}^h + S^h}$$

Note, that we use a formulation of the Hill equation, where the constant on the denominator is raised to the power of h. Therefore, the constant also marks the half-way saturation concentration and we denote it $K_{\frac{1}{2}}$

As usual start by writing a function implementing this equation

```
def hill_eq(S, Vmax, K, h):
    ## Your task: Implement the enzyme kinetics Hill equation.
    return (Vmax * S**h) / (K**h + S**h)

print(hill_eq(1, 1, 1, 1)) # Should give 0.5 -> (1 * 1^1) / (1**1 + 1**1) = (1 * 1) / (1 +
1) = 1/2
```

```
0.5
```

And then make the fit for each dataset of rates vs. aspartate concentration.

```
## Your task: Make the fit for the dataset without CTP
fitted_parameters_noctp, _ = curve_fit(hill_eq, df['[aspartate]_(M)'], df['rate_(M/s)'])
Vmax_fit_noctp, K_fit_noctp, h_fit_noctp = fitted_parameters_noctp

## Your task: Make the fit for the dataset with CTP
fitted_parameters_ctp, _ = curve_fit(hill_eq, df['[aspartate]_(M)'], df['rate_ctp_(M/s)'])
Vmax_fit_ctp, K_fit_ctp, h_fit_ctp = fitted_parameters_ctp
```

The next cell prints the fitted parameters for both cases.

```
print('Without CTP')
print('    Vmax', Vmax_fit_noctp)
print('    K1/2', K_fit_noctp)
print('    h', h_fit_noctp)

print('With CTP')
print('    Vmax', Vmax_fit_ctp)
```

```
print('    K1/2', K_fit_ctp)
print('    h', h_fit_ctp)
```

```
Without CTP
    Vmax 1.8001107308873426e-05
    K1/2 0.009189687811902407
    h 2.388780830152653
With CTP
    Vmax 1.799574252747415e-05
    K1/2 0.013868061305082755
    h 2.698543683234914
```

Then we can plot

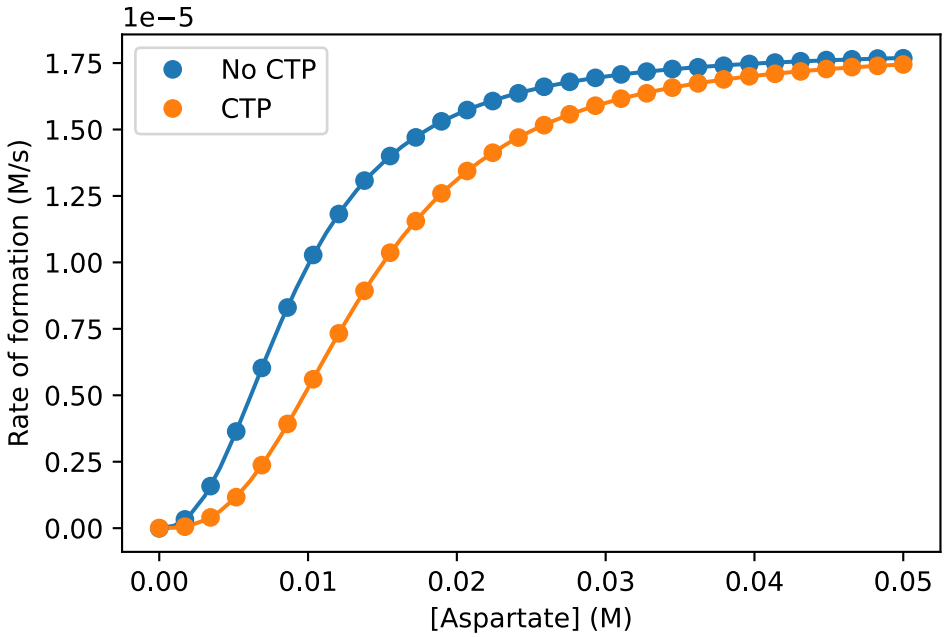
```
fig, ax = plt.subplots()

## Your task: Calculate the fits
S_smooth = np.linspace(0, 50*10**(-3))
V_fit_noctp = hill_eq(S_smooth, Vmax_fit_noctp, K_fit_noctp, h_fit_noctp)
V_fit_ctp = hill_eq(S_smooth, Vmax_fit_ctp, K_fit_ctp, h_fit_ctp)

## Your task: Plot the fits:
ax.plot(S_smooth, V_fit_noctp)
ax.plot(S_smooth, V_fit_ctp)

## Plots the datasets
ax.plot(df['[aspartate]_(M)'], df['rate_(M/s)'], 'o', label='No CTP', color='C0')
ax.plot(df['[aspartate]_(M)'], df['rate_ctp_(M/s)'], 'o', label='CTP', color='C1')

## Labels & legend
ax.set_xlabel('[Aspartate] (M)')
ax.set_ylabel('Rate of formation (M/s)')
ax.legend()
```



How do the constants of the Hill equation change in response to the presence of CTP?
Does this match your expectations?

! Answer

V_{max} is basically unchanged.

$K_{} is increased meaning that a higher aspartate concentration is necessary in order to reach half V_{max} - this matches with the curve being shifted to the right.

h increases slightly in the presence of CTP making the curve more sigmoidal, so the switch is sharper.